

ITRF2020 到其余 ITRF 框架转换报告

一、运行方式

运行环境：VS2022 C#

可运行程序文件：ITRF2020 到其余 ITRF 框架转换代码\bin\Debug\

ITRFCoordinateTransformation.exe

执行方式：

1. 选择目标框架和历元；
2. 输入待转换的坐标点；
3. 开始转换。

二、原理

不同的 ITRF 框架之间可通过七参数转换模型进行转换，这七个参数包括三个平移参数、三个旋转参数和一个尺度参数，用于描述两个坐标系之间的差异。其转换公式如下：

$$\begin{bmatrix} X_S \\ Y_S \\ Z_S \end{bmatrix} = \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} + \begin{bmatrix} T_x \\ T_y \\ T_z \end{bmatrix} + \begin{bmatrix} D & -R_z & R_y \\ R_z & D & -R_x \\ -R_y & R_x & D \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

其中，X,Y,Z 是 ITRF2020 框架下的坐标，X_S,Y_S,Z_S是其他 ITRF 框架下的坐标，T_x,T_y,T_z是平移参数，R_x,R_y,R_z是旋转参数，D 是尺度参数。

由于地壳运动等因素，ITRF 框架的转换参数并不是固定不变

的，而是会随时间发生变化。在任意时刻的 7 个参数需要考虑变化速率，要先解算出对应历元两个框架之间的 7 个转换参数。对于给定参数 P ，其在任意历元 t 的值可通过公式以下公式计算：

$$P(t) = P(EPOCH) + \dot{P} \times (t - EPOCH)$$

其中 $EPOCH$ 是给定的参考历元， $\dot{P}$ 是参数 P 的变化速率。

一般先进行历元转换，再进行框架转换。历元转换是因为 GPS 测站在框架内的位置随时间近似匀速线性变化，根据坐标变化速率将坐标转换到统一历元；然后再利用对应历元下的七参数，按照布尔莎-沃尔夫七参数转换模型进行框架转换，从而实现不同 ITRF 框架和历元下坐标的转换。

三、代码

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
namespace ITRFCoordinateTransformation
{
    public partial class ITRFTransformForm : Form
    {
        private ITRFTransformer transformer;
        public ITRFTransformForm()
        {
            InitializeComponent();
            transformer = new ITRFTransformer();
            // 设置默认选择项
            cboTargetFrame.SelectedIndex = 0;
        }
    }
}
```

```

    }

    private void btnTransform_Click(object sender, EventArgs e)
    {
        try
        {
            string targetFrame = cboTargetFrame.SelectedItem.ToString();
            // 清空输出表格
            dgvOutput.Rows.Clear();
            // 遍历输入表格的每一行
            for (int i = 0; i < dgvInput.Rows.Count - 1; i++)
            {
                // 获取输入值
                string pointNumber = dgvInput.Rows[i].Cells["dataGridViewTextBoxColumn4"].Value?.ToString();
                double x = double.Parse(dgvInput.Rows[i].Cells["dataGridViewTextBoxColumn5"].Value.ToString()
);
                double y = double.Parse(dgvInput.Rows[i].Cells["dataGridViewTextBoxColumn6"].Value.ToString()
);
                double z = double.Parse(dgvInput.Rows[i].Cells["dataGridViewTextBoxColumn7"].Value.ToString()
);

                double epoch = double.Parse(txtEpoch.Text);
                // 执行坐标转换
                double[] result = transformer.TransformCoordinates(x, y, z, targetFrame, epoch);
                // 将结果添加到输出表格
                dgvOutput.Rows.Add(pointNumber, result[0].ToString("F6"), result[1].ToString("F6"), result[2].ToSt
ring("F6"));
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show($"转换过程中发生错误: {ex.Message}", "错误
", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }

    private void lblResult_Click(object sender, EventArgs e)
    {
    }

    private void ITRFTransformForm_Load(object sender, EventArgs e)
    {
    }

    private void btnClear_Click(object sender, EventArgs e)
    {
        // 清空输入表格
        dgvInput.Rows.Clear();
        // 清空输出表格

```

```

        dgvOutput.Rows.Clear();
    }
    private void lblTitle_Click(object sender, EventArgs e)
    {
    }
}

public class ITRFTransformer
{
    // 转换参数数据结构:
    [Tx, Ty, Tz, D, Rx, Ry, Rz, epoch, rate_Tx, rate_Ty, rate_Tz, rate_D, rate_Rx, rate_Ry, rate_Rz]
    // 单位:
    mm, mm, mm, ppb, 0.001", 0.001", 0.001", year, mm/y, mm/y, mm/y, ppb/y, 0.001"/y, 0.001"/y, 0.001"/y

    private Dictionary<string, double[]> transformationParams;
    public ITRFTransformer()
    {
        // 初始化转换参数
        transformationParams = new Dictionary<string, double[]>
        {
            { "ITRF2014", new double[] { -1.4, -0.9, 1.4, -0.42, 0.00, 0.00, 0.00, 2015.0, 0.0, -
0.1, 0.2, 0.00, 0.00, 0.00, 0.00 } },
            { "ITRF2008", new double[] { 0.2, 1.0, 3.3, -0.29, 0.00, 0.00, 0.00, 2015.0, 0.0, -
0.1, 0.1, 0.03, 0.00, 0.00, 0.00 } },
            { "ITRF2005", new double[] { 2.7, 0.1, -1.4, 0.65, 0.00, 0.00, 0.00, 2015.0, 0.3, -
0.1, 0.1, 0.03, 0.00, 0.00, 0.00 } },
            { "ITRF2000", new double[] { -0.2, 0.8, -34.2, 2.25, 0.00, 0.00, 0.00, 2015.0, 0.1, 0.0, -
1.7, 0.11, 0.00, 0.00, 0.00 } },
            { "ITRF97", new double[] { 6.5, -3.9, -77.9, 3.98, 0.00, 0.00, 0.36, 2015.0, 0.1, -0.6, -
3.1, 0.12, 0.00, 0.00, 0.02 } },
            { "ITRF96", new double[] { 6.5, -3.9, -77.9, 3.98, 0.00, 0.00, 0.36, 2015.0, 0.1, -0.6, -
3.1, 0.12, 0.00, 0.00, 0.02 } },
            { "ITRF94", new double[] { 6.5, -3.9, -77.9, 3.98, 0.00, 0.00, 0.36, 2015.0, 0.1, -0.6, -
3.1, 0.12, 0.00, 0.00, 0.02 } },
            { "ITRF93", new double[] { -65.8, 1.9, -71.3, 4.47, -3.36, -4.33, 0.75, 2015.0, -2.8, -0.2, -2.3, 0.12, -
0.11, -0.19, 0.07 } },
            { "ITRF92", new double[] { 14.5, -1.9, -85.9, 3.27, 0.00, 0.00, 0.36, 2015.0, 0.1, -0.6, -
3.1, 0.12, 0.00, 0.00, 0.02 } },
            { "ITRF91", new double[] { 26.5, 12.1, -91.9, 4.67, 0.00, 0.00, 0.36, 2015.0, 0.1, -0.6, -
3.1, 0.12, 0.00, 0.00, 0.02 } },
            { "ITRF90", new double[] { 24.5, 8.1, -107.9, 4.97, 0.00, 0.00, 0.36, 2015.0, 0.1, -0.6, -
3.1, 0.12, 0.00, 0.00, 0.02 } },
            { "ITRF89", new double[] { 29.5, 32.1, -145.9, 8.37, 0.00, 0.00, 0.36, 2015.0, 0.1, -0.6, -
3.1, 0.12, 0.00, 0.00, 0.02 } },
            { "ITRF88", new double[] { 24.5, -3.9, -169.9, 11.47, 0.10, 0.00, 0.36, 2015.0, 0.1, -0.6, -
3.1, 0.12, 0.00, 0.00, 0.02 } }
        }
    }
}

```

```

    };
}

public double[] TransformCoordinates(double x, double y, double z, string targetFrame, double epoch)
{
    if (!transformationParams.ContainsKey(targetFrame))
    {
        throw new ArgumentException($"不支持的 ITRF 框架: {targetFrame}");
    }

    double[] paramsArray = transformationParams[targetFrame];
    // 提取参数和速率

    double tx0 = paramsArray[0];
    double ty0 = paramsArray[1];
    double tz0 = paramsArray[2];
    double d0 = paramsArray[3];
    double rx0 = paramsArray[4];
    double ry0 = paramsArray[5];
    double rz0 = paramsArray[6];
    double refEpoch = paramsArray[7];

    double txRate = paramsArray[8];
    double tyRate = paramsArray[9];
    double tzRate = paramsArray[10];
    double dRate = paramsArray[11];
    double rxRate = paramsArray[12];
    double ryRate = paramsArray[13];
    double rzRate = paramsArray[14];
    // 计算时间差

    double dt = epoch - refEpoch;
    // 根据公式  $P(t) = P(EPOCH) + P\_rate * (t - EPOCH)$  计算参数

    double tx = tx0 + txRate * dt;
    double ty = ty0 + tyRate * dt;
    double tz = tz0 + tzRate * dt;
    double d = d0 + dRate * dt;
    double rx = rx0 + rxRate * dt;
    double ry = ry0 + ryRate * dt;
    double rz = rz0 + rzRate * dt;
    // 转换单位: 将微弧秒转换为弧度

    double rxRad = rx * (0.001 / 3600.0) * (Math.PI / 180.0);
    double ryRad = ry * (0.001 / 3600.0) * (Math.PI / 180.0);
    double rzRad = rz * (0.001 / 3600.0) * (Math.PI / 180.0);
    // 将尺度参数从 ppb 转换为实际比例因子

    double dScale = 1.0 + d * 1e-9;
    // 将参数转换为米

    double txM = tx / 1000.0; // 从 mm 转换为 m
    double tyM = ty / 1000.0;

```

```
double tzM = tz / 1000.0;
// 构建旋转矩阵
double[,] rotationMatrix = new double[3, 3]
{
    { dScale, -rzRad, ryRad },
    { rzRad, dScale, -rxRad },
    { -ryRad, rxRad, dScale }
};
// 原始坐标向量
double[] originalVector = new double[] { x, y, z };
// 应用旋转
double[] rotatedVector = new double[3];
for (int i = 0; i < 3; i++)
{
    for (int j = 0; j < 3; j++)
    {
        rotatedVector[i] += rotationMatrix[i, j] * originalVector[j];
    }
}
// 应用平移
double[] transformedVector = new double[3];
for (int i = 0; i < 3; i++)
{
    transformedVector[i] = rotatedVector[i] + new double[] { txM, tyM, tzM }[i];
}
return transformedVector;
}
}
```

四、结果

初始界面：

ITRF坐标转换工具 - 张洋20232411

ITRF2020到历史框架坐标转换工具

目标ITRF框架: ITRF2014

历元: 2020.0

开始转换

清空

	点号	X	Y	Z
*				

	点号	X	Y	Z
*				

选择目标框架，输入目标历元：

ITRF坐标转换工具 - 张洋20232411

ITRF2020到历史框架坐标转换工具

目标ITRF框架: ITRF2014

历元: 2020.0

开始转换

清空

	点号	X	Y	Z
*				

	点号	X	Y	Z
*				

转换示例：

ITRF坐标转换工具 - 张洋20232411

ITRF2020到历史框架坐标转换工具

目标ITRF框架: ITRF91

历元: 2018

开始转换

清空

	点号	X	Y	Z
	1	1	1	1
▶	2	30	25	90
*				

	点号	X	Y	Z
▶	1	1.026800	1.010300	0.898800
	2	30.026800	25.010300	89.898800
*				